

Rapport de Projet PEIP-2

Réalisation d'un logiciel pour la détection de pannes dans des systèmes d'énergie renouvelable et de son interface graphique (GUI).

Noah RAIMBAUD, Louis ARNAUD

2023-2024

Équipe de Projet

- Noah RAIMBAUD
- Louis ARNAUD

Encadrement

- Bassel CHOKR
- Nizar CHATT



Remerciements

Nous tenons à remercier nos encadrants, Bassel CHOKR et Nizar CHATTI qui ont accompagné notre travail, en nous laissant de l'autonomie pour la réalisation de ce projet et nos apprentissages.

Table des matières

1	Introduction	3
2	Travail réalisé	4
2.1	Présentation du cahier des charges fonctionnel	4
2.2	Description du travail réalisé	5
2.3	Les problèmes rencontrés et comment nous y avons fait face	15
2.4	Organisation du temps de travail et répartition des tâches	16
3	Apprentissages	17
3.1	La réalisation d'une interface graphique	17
3.2	Introduction aux algorithmes d'IA	18
3.3	Bibliothèques Python	19
4	Conclusion	19
4.1	Critique des résultats obtenus et pistes d'approfondissement	19
4.2	Conclusion générale	19
4.3	Conclusion personnelle de Louis ARNAUD	20
4.4	Conclusion personnelle de Noah RAIMBAUD	20
5	Annexes	21

1 Introduction

Le présent rapport rend compte du projet de conception réalisé dans le cadre de notre deuxième année d'études en classe préparatoire, à Polytech Angers. Il représente plus de 100 heures de travail pendant lesquelles nous avons développé le projet et rédigé un article de blog, ainsi que ce rapport.

Suite à l'avènement des énergies renouvelables, anticiper le comportement défectueux des systèmes est devenu un réel enjeu. Réussir à prédire de futures pannes, grâce à l'acquisition et au traitement de données, provenant des systèmes en temps réel, favoriserait une maintenance préventive plutôt que curative. Les avantages de cette méthode sont nombreux : la réduction des coûts de maintenance et du temps de réparation, un meilleur entretien des matériels et l'optimisation de la production d'énergie.

L'utilisation d'algorithmes de prédiction et d'apprentissage supervisé (qui consistent à analyser des données "labelisées" au préalable) apporte une réponse à ce besoin. Cependant, de nombreuses étapes préliminaires sont nécessaires pour les utiliser, ce qui peut les rendre complexes à appréhender. En effet, pour appliquer les algorithmes d'IA à des systèmes d'énergie renouvelable il est nécessaire d'utiliser des techniques d'analyse des données. Les fichiers de mesure provenant des systèmes d'énergie renouvelable contiennent énormément de caractéristiques dont certaines ne sont pas pertinentes pour l'apprentissage des algorithmes. Des techniques de réduction de dimensions, de normalisation et de standardisation des données sont alors nécessaires pour optimiser l'utilisation des algorithmes. Nous avons utilisé une technique nommée "feature selection" ou sélection de caractéristiques en français. Cela consiste à éliminer ou non certaines caractéristiques en fonction de leur pertinence pour la classification des données. Dans la même optique de simplification des données, l'approche d'analyse en composantes principales (PCA) est également un outil puissant.

Une fois toutes ces contraintes traitées, les algorithmes de prédiction deviennent utilisables. Il en existe plusieurs (nous en avons choisi quatre) qui ont tous un fonctionnement différent et des performances inégales. La performance de chaque algorithme varie également en fonction des hyperparamètres qui sont utilisés pour leur fonctionnement. Pour choisir l'algorithme et ses hyperparamètres les plus adaptés, il faut évaluer les performances de chaque modèle, ce qui, encore une fois, n'est pas chose aisée. Des métriques (nommées "accuracy", "precision", "recall" et "F1-score") permettent de mesurer cette performance. L'étude comparative de ces métriques permet de choisir le meilleur algorithme.

Notre binôme a développé un logiciel et son interface graphique répondant à ce besoin. Son intérêt est de rendre accessible l'usage, pourtant complexe, d'algorithmes d'apprentissage supervisé, via une interface intuitive et simple d'utilisation. C'est également un logiciel qui se veut générique, c'est à dire utilisable avec des données provenant de systèmes énergétiques ou d'équipements de production industrielle. Enfin, il est un outil d'aide à la décision qui accompagne l'utilisateur dans le choix du meilleur algorithme.

Ce projet a été riche en apprentissages. Nous avons appris, dans les grandes lignes, comment fonctionnent plusieurs algorithmes d'apprentissage supervisé. Nous avons également développé nos compétences informatiques et nous nous sommes familiarisés avec le langage de programmation Python ainsi qu'avec pléthore de ses bibliothèques.

Nous présenterons dans un premier temps le travail réalisé de la façon la plus exhaustive possible. Pour cela, nous indiquerons le cahier des charges fonctionnel de notre logiciel puis nous décrirons en détail ses fonctionnalités. Nous discuterons également des problèmes que nous avons rencontré et de leur résolution, ainsi que de l'organisation de notre travail. Dans un deuxième temps, nous aborderons ce que notre projet nous a apporté et appris. Enfin, nous apporterons un regard critique sur notre réalisation et proposerons des pistes d'amélioration et d'approfondissement.

2 Travail réalisé

2.1 Présentation du cahier des charges fonctionnel

a) Importation des données	— Importer un fichier de données (CSV/XLSX/PARQUET) — Fonctionnalité "drag and drop" — Fonctionnalité ouvrant l'explorateur de fichier
b) Visualisation des données	— Visualisation des données dans un tableau (ressemblant au fichier d'origine) — Visualisation graphique de différentes natures (courbe, histogramme, boîte à moustache, nuage de point). — Visualisation de données statistiques (moyenne et à l'écart-type) pour chaque caractéristique (colonne).
c) Préparation des données.	— Fonctionnalité de suppression de colonnes — Normalisation/standardisation des données — Séparation des données en deux groupes (d'entraînement et de test) — Sélection de la proportion entraînement/test
d) Feature Engineering	— Fonctionnalité "Feature Selection" — Affichage de l'importance de chaque caractéristique = colonne de mesure — Renseigner un seuil d'importance — Supprimer les colonnes dont l'importance est inférieure au seuil sélectionné — Fonctionnalité PCA — Renseigner un pourcentage d'information à conserver — Supprimer les colonnes qui ne sont pas nécessaires pour avoir le pourcentage d'information renseigné
e) Utilisation des algorithmes d'IA	— Possibilité d'utiliser 4 algorithmes différents : — Decision Tree Classifier (CART) — K-Nearest Neighbors (KNN) — Random Forest — Multi-layer Perceptron (MLP) — Modification des hyperparamètres — Affichage de métriques (F1 score, accuracy...) — Affichage de matrices de confusion d'entraînement et de test
f) Évaluation des algorithmes	— Possibilité de comparer plusieurs algorithmes entre eux. — Affichage de plusieurs métriques d'évaluation ("accuracy", "precision", "F1-score", "recall") — Exporter le modèle retenu

TABLE 1 – Tableau de cahier des charges fonctionnel

Les différentes étapes présentées sont également décrites par ce diagramme :

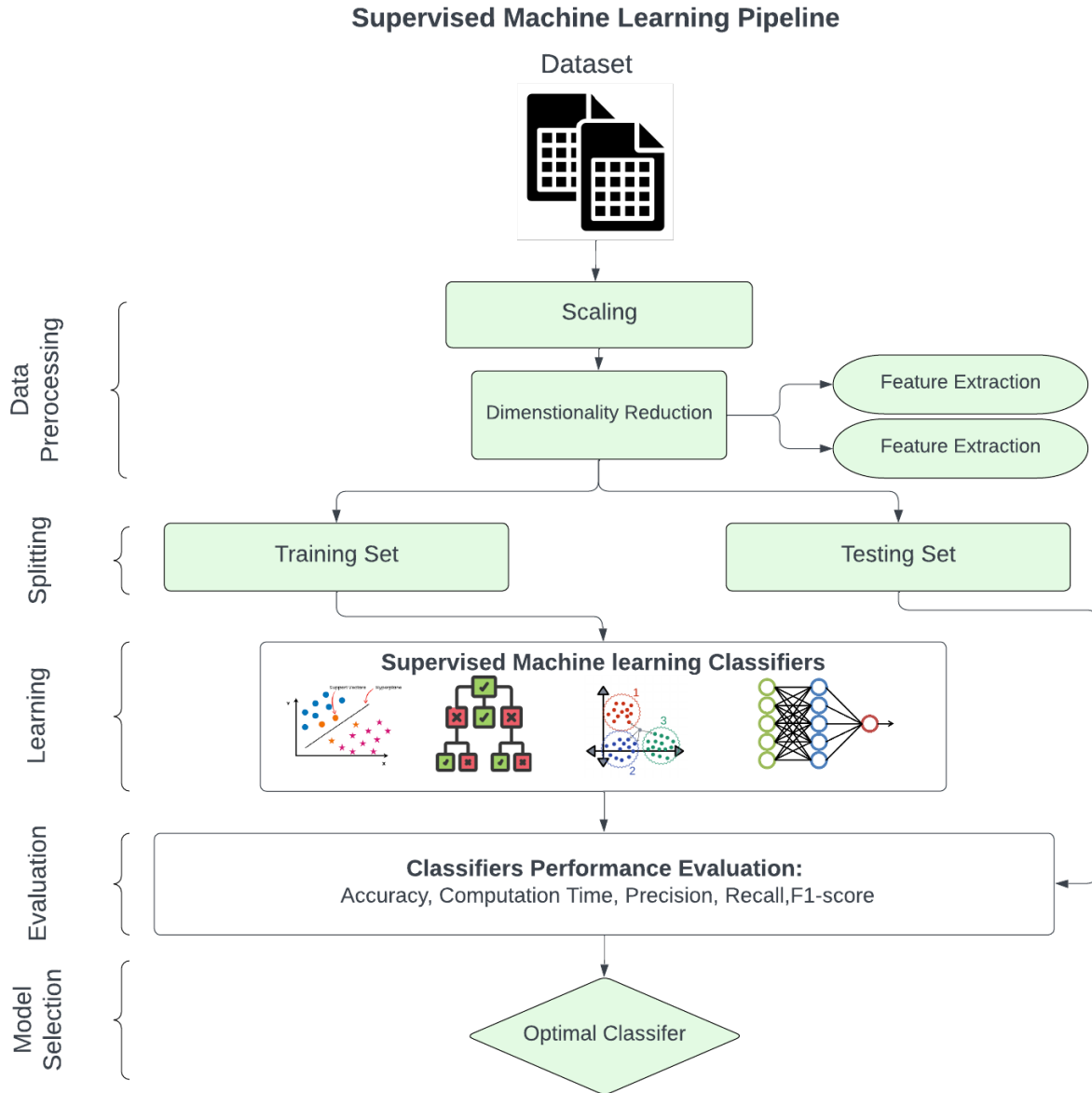


FIGURE 2 – Pipeline d'apprentissage supervisé

2.2 Description du travail réalisé

Nous présentons dans cette partie le logiciel que nous avons développé. Ce logiciel comporte de multiples fonctionnalités (cf. cahier des charges fonctionnel). Nous allons ici éclaircir leur fonctionnement et leur utilité.

Le logiciel fonctionne par étapes (voir barre de progression en haut du logiciel). Une fois qu'une étape est accomplie, l'utilisateur est invité à passer à la suivante.

Les étapes en question sont :

a) Importation des données

L'utilisateur importe un fichier de données (excel, CSV, parquet). Il peut le faire de plusieurs façons. Il peut en effet renseigner le chemin d'accès du fichier qu'il veut ouvrir ou utiliser la fonctionnalité "glisser déposer" (drag and drop). Enfin, une dernière fonctionnalité permet d'ouvrir directement les deniers fichiers utilisés, afin de gagner du temps.

Ces fonctionnalités se retrouvent sur la page d'accueil, que nous avons voulue simple et agréable vi-

suellement. Plusieurs images d'illustration peuvent être affichées aléatoirement aux côtés du titre. La zone d'importation peut recevoir les fichiers en "glisser déposer". Lorsque l'on clique dessus, l'explorateur de fichiers de l'ordinateur s'ouvre pour sélectionner manuellement le fichier. Enfin, nous avons crédité (en bas de page) nos tuteurs, l'école ainsi que le laboratoire LARIS.

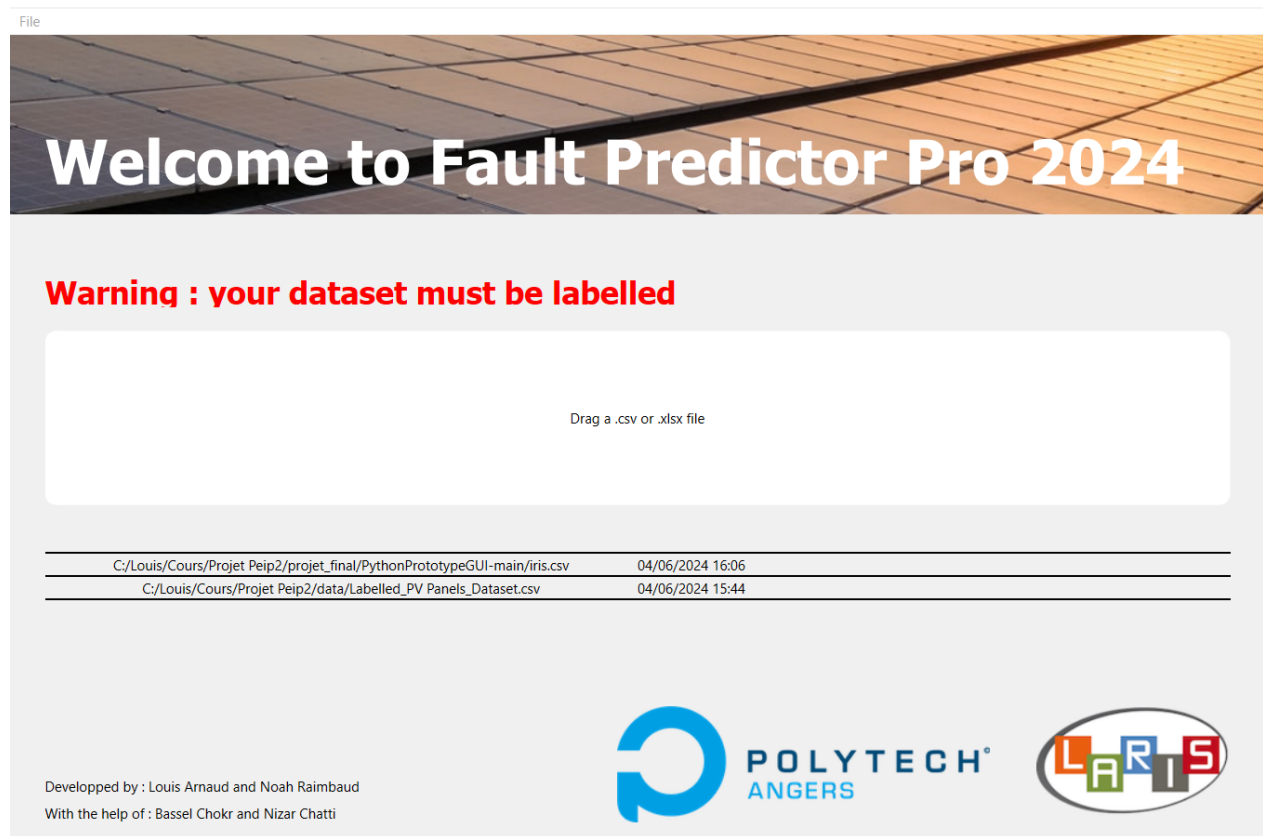


FIGURE 3 – page d'accueil et d'importation

b) Visualisation des données

Après avoir importé un fichier dans la page accueil, le logiciel passe à la partie "configuration du modèle de machine learning". La fenêtre se compose de trois éléments : une barre de visualisation de la progression (au dessus), le contenu de l'étape (au milieu), et la barre de navigation (en bas de page). La barre de progression montre l'avancement des étapes pour la configuration du modèle. La barre de navigation contient un bouton "étape suivante", un bouton "étape précédente" et peut afficher, si nécessaire, une barre de chargement pour visualiser l'avancement des calculs importants.

Cette configuration du modèle commence par la page "Visualisation des données" qui propose 3 types de visualisation. Tout d'abord, l'utilisateur peut lire les données dans un tableau, à la manière d'un logiciel de type "tableur". En raison d'un problème cité dans la partie suivante "Les problèmes rencontrés", nous avons limité l'affichage des données aux 1000 premiers éléments du fichier. Cela permet de donner un aperçu des données sans passer plusieurs minutes à générer le tableau.

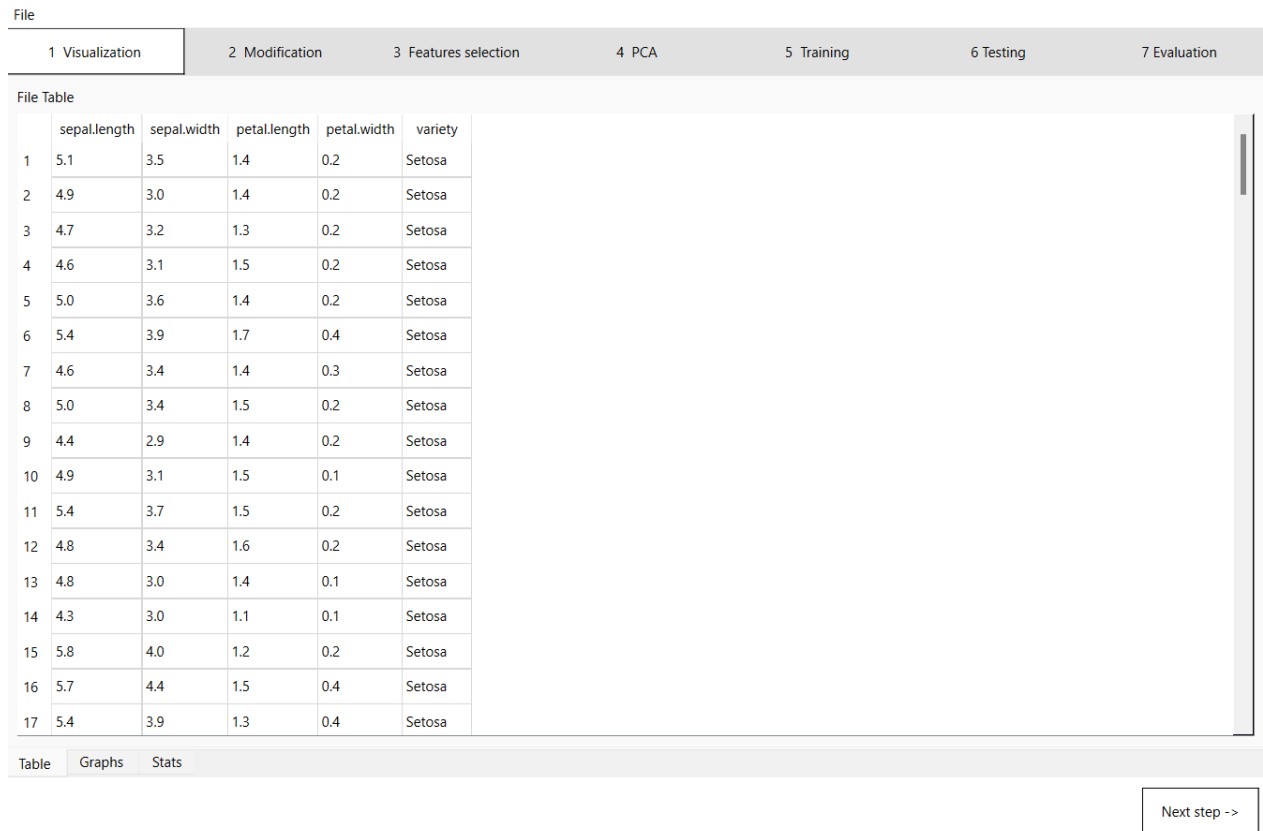


FIGURE 4 – page de visualisation

L'utilisateur peut également visualiser les données à travers des graphiques de différentes natures (courbe, histogramme, boîte à moustache, nuage de points). Chaque type d'affichage donne une approche de visualisation différente pour aider l'utilisateur à comprendre la structuration générale des données qu'il utilise. L'utilisateur peut choisir les variables qu'il veut afficher en abscisse et en ordonnée. Ces fonctionnalités font de notre logiciel un outil de visualisation de données intuitif.

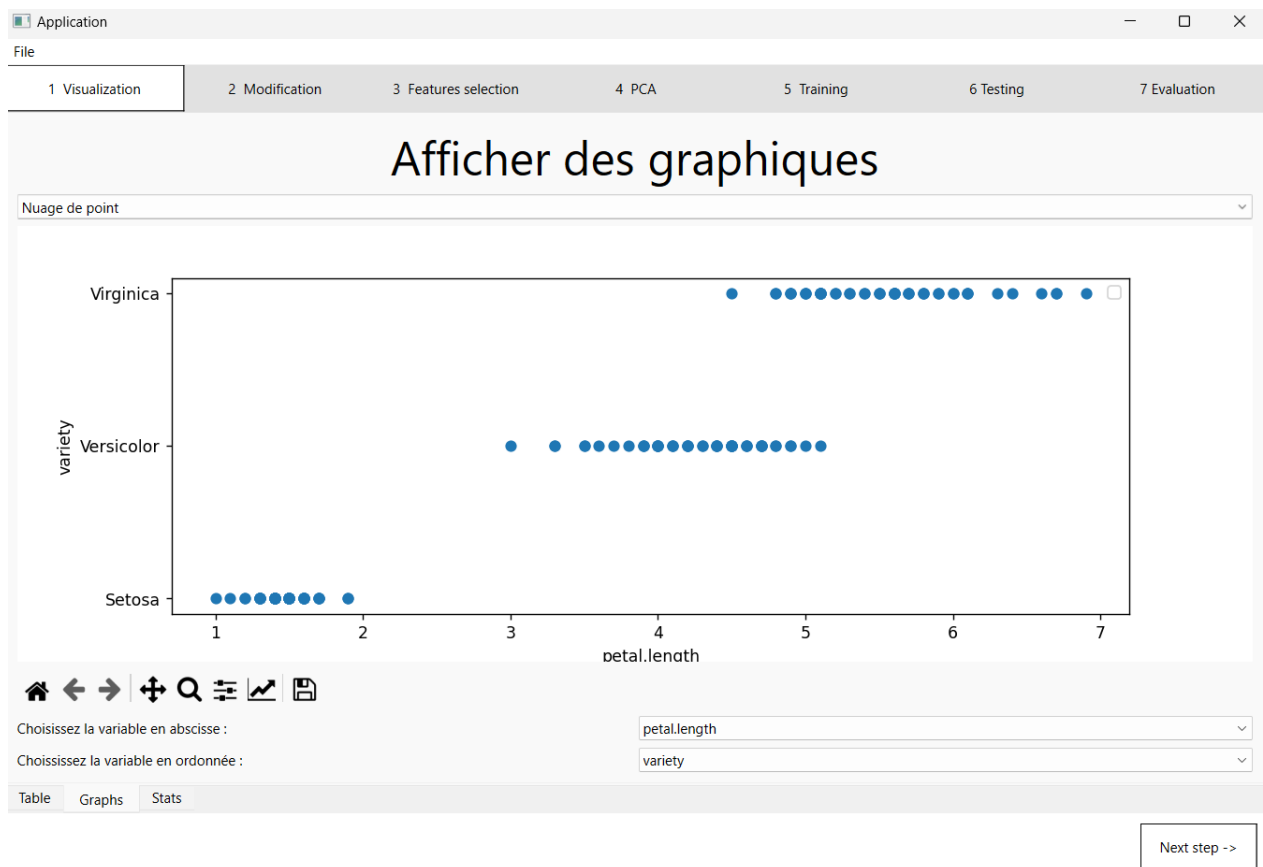


FIGURE 5 – page d’affichage graphique

Enfin, il peut visualiser la moyenne et l’écart-type de des données de chaque caractéristique de données. Une matrice de corrélation est également affichée, explicitant le rapport qu’il peut y avoir entre certaines caractéristiques (1 exprime une corrélation parfaite, -1 exprime une contradiction totale).



FIGURE 6 – page d’affichage statistique

c) Préparation des données

Cette étape permet à l'utilisateur de modifier et de préparer les données. Une fonctionnalité permet la suppression manuelle de colonnes du fichier qui seraient inutiles. L'utilisateur peut aussi normaliser ou standardiser les colonnes du fichier. Ainsi, les valeurs numériques du fichier sont mises à la même échelle afin qu'aucune colonne n'occupe un poids plus important qu'une autre dans les algorithmes utilisés par la suite.

Formule pour la standardisation :

$$Z = \frac{x_i - \mu}{\sigma}$$

Formule pour la normalisation (min-max scaling) :

$$x' = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}$$

Un bouton permet d'exporter le fichier modifié, en format CSV. On peut donc se servir de notre logiciel comme outil de modification de données.

Pour aller plus loin, l'utilisateur doit séparer les données en deux sous ensembles. L'un servira à entraîner les algorithmes d'IA, l'autre à les tester (l'utilisateur peut choisir le pourcentage de données qu'il veut allouer à l'ensemble de test et à l'ensemble d'entraînement). Une fois que cela est fait, l'utilisateur peut passer à l'étape suivante.

Afin de rendre l'expérience plus agréable pour l'utilisateur, nous avons ajouté un bouton de retour en arrière qui permet à chaque modification des données de revenir à la précédente version du tableau, par exemple en cas d'erreur.

Nous avons regroupé les éléments d'interaction sur la partie droite de la page. Afin de rendre cela plus compréhensible, chaque bouton a été numéroté pour expliciter le processus de modification des données. À l'ouverture de la page, seule l'étape 1 "delete column" et l'étape 2 "split data" sont disponibles. Les

autres boutons sont "grisés", c'est à dire bloqués. Ils ne se débloquent que lorsque l'utilisateur progresse dans les étapes. Cela permet d'empêcher les erreurs en garantissant que les étapes sont effectuées dans l'ordre nécessaire au bon fonctionnement du code.

Les données sont quant à elles affichées sur la gauche, exactement comme sur la page précédente. La raison est simple, nous avons isolé certains composants de la page afin de pouvoir les réutiliser dans plusieurs pages. Et c'est le cas de ce tableau d'affichage.

Le défi était de pouvoir visualiser les changements apportés au fichier lorsque l'utilisateur modifie les données. Afin de gagner du temps, nous avons décidé de régénérer le tableau dans son entièreté à chaque modification. À l'étape 2, pour montrer la séparation du fichier en deux parties, la page affiche deux tableaux distincts (un d'entraînement et un de test).

Cette page a beaucoup évolué. À l'origine, une fonctionnalité permettait à l'utilisateur de modifier n'importe quelle cellule du tableau. Malheureusement, cette fonctionnalité occasionnait trop d'erreurs dans le logiciel. Nous avons donc décidé de la supprimer qui en outre n'était pas essentielle.

	sepal.length	sepal.width	petal.length	petal.width	variety
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
3	4.7	3.2	1.3	0.2	Setosa
4	4.6	3.1	1.5	0.2	Setosa
5	5.0	3.6	1.4	0.2	Setosa
6	5.4	3.9	1.7	0.4	Setosa
7	4.6	3.4	1.4	0.3	Setosa
8	5.0	3.4	1.5	0.2	Setosa
9	4.4	2.9	1.4	0.2	Setosa
10	4.9	3.1	1.5	0.1	Setosa
11	5.4	3.7	1.5	0.2	Setosa
12	4.8	3.4	1.6	0.2	Setosa
13	4.8	3.0	1.4	0.1	Setosa
14	4.3	3.0	1.1	0.1	Setosa
15	5.8	4.0	1.2	0.2	Setosa
16	5.7	4.4	1.5	0.4	Setosa
17	5.4	3.9	1.3	0.4	Setosa
18	5.1	3.5	1.4	0.3	Setosa

FIGURE 7 – page "modification"

d) Analyse des données et aide à la décision pour l'abandon des caractéristiques peu importantes

Dans les fichiers de données des panneaux photo-voltaïques et d'éolienne, beaucoup de mesures sont retranscrites. Cependant, certaines d'entre elles ne sont pas utiles à la détection de pannes par les algorithmes d'IA. Grâce à un indicateur nommé « information gain », il est possible de mesurer la pertinence de chaque caractéristique pour la classification des points de données. Cette méthode est basée sur l'entropie :

$$E(y) = - \sum C_i p_i \log_2(p_i)$$

Dans notre logiciel, un graphique en barre permet d'afficher l'importance de chaque colonnes. L'utilisateur renseigne ensuite un seuil d'importance à l'aide d'un curseur. Les caractéristiques dont l'importance des données est inférieure à ce seuil ne seront pas prises en compte par la suite (ces colonnes sont affichées en

rouge sur l'image ci-dessous). Grâce au graphique affiché avec la barre de seuil et le curseur, l'utilisateur peut facilement visualiser les caractéristiques supprimées.

L'intérêt de la suppression de caractéristiques est d'économiser du temps de calcul lors de l'entraînement du modèle de machine learning. Moins de paramètres pris en compte, c'est moins de calculs sans pour autant trop perdre en précision.

À l'origine, la sélection de la valeur du seuil se faisait dans une zone où l'utilisateur pouvait taper manuellement la valeur. Mais pour rendre l'interaction plus naturelle, nous avons ajouté ce curseur. Nous avons également effectué cette modification sur la page suivante.

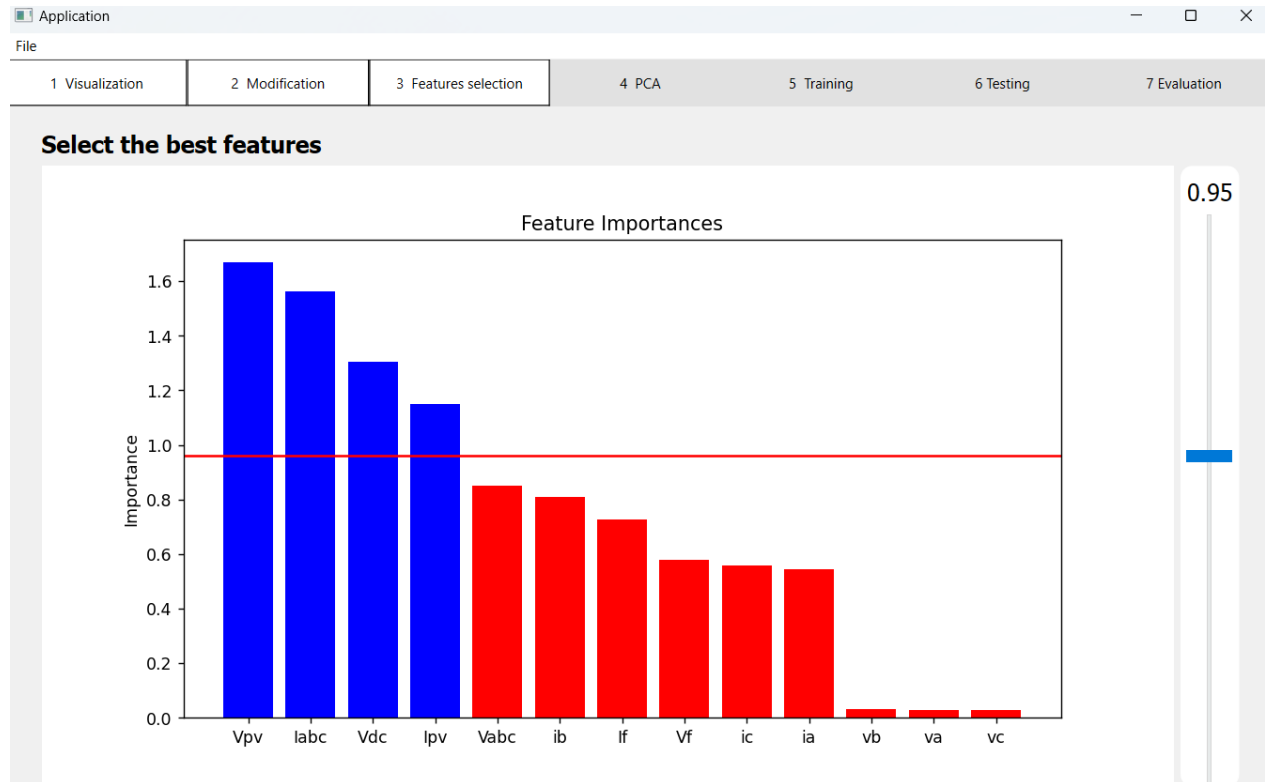


FIGURE 8 – page "Feature Selection"

Les données sont à nouveau modifiées dans une étape appelée PCA (Principal Component Analysis). Elle requiert des données mises à l'échelle (normalisées ou standardisées), d'où l'importance de l'effectuer à l'étape précédente. C'est une méthode d'analyse des données qui consiste à transformer des variables corrélées en de nouvelles variables décorréliées les unes des autres. On appelle les variables ainsi obtenues « composantes principales ». Elle permet de résumer l'information en réduisant le nombre de variables. L'utilisateur entre alors le pourcentage d'information qu'il veut garder pour utiliser les algorithmes d'IA. La sélection du seuil se fait à nouveau à l'aide d'un curseur et d'une barre de seuil sur le graphique. Un nombre de composantes principales à utiliser est alors choisi. Cette étape permet à nouveau de réduire la quantité de calcul pour un fonctionnement optimal des algorithmes. Le logiciel empêche l'utilisateur de supprimer l'entièreté des données, ce qui ferait planter le logiciel.

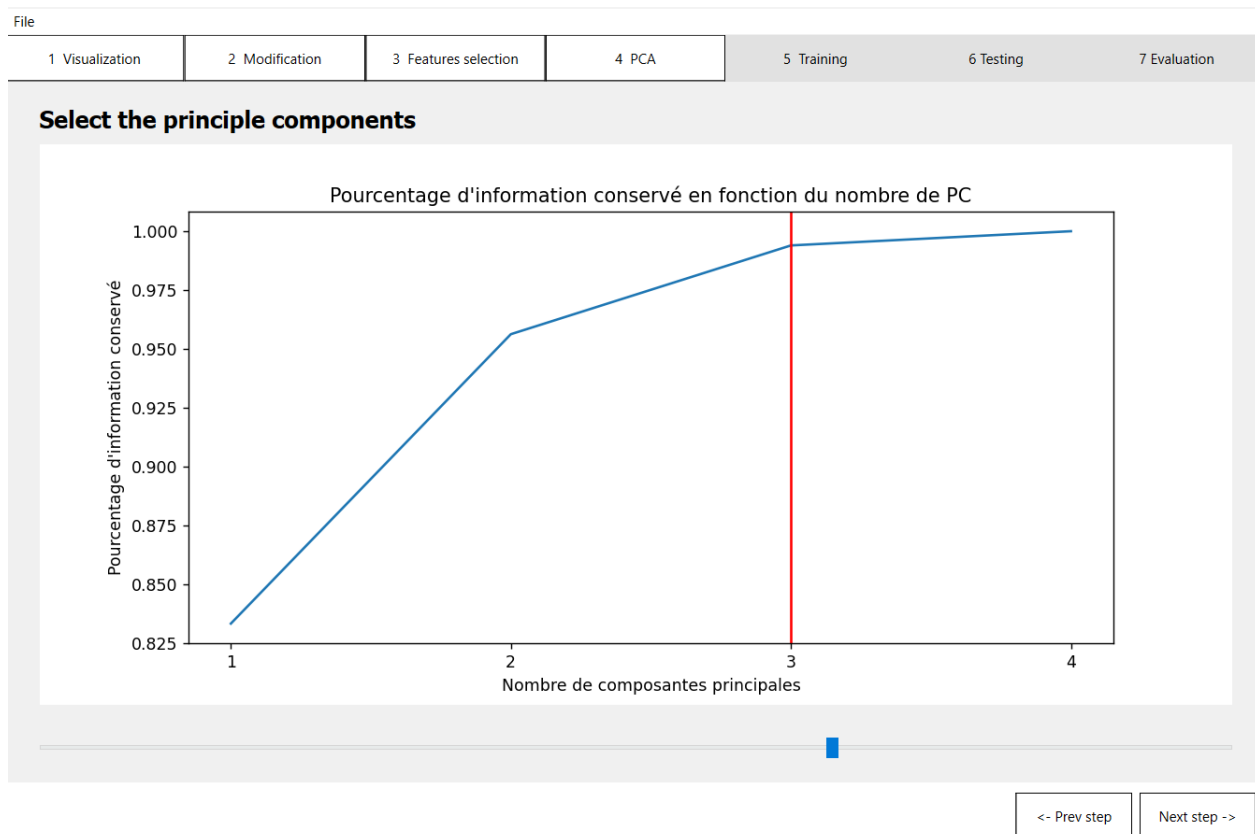


FIGURE 9 – page "PCA"

e) Utilisation des algorithmes d'intelligence artificielle

Grâce à la "feature selection" et au "PCA" nous obtenons un ensemble de données prêt à être utilisé dans les algorithmes d'IA. Notre projet repose sur l'apprentissage supervisé. C'est à dire que les données utilisées pour entraîner les algorithmes de prédiction doivent être labelisées (étiquetées).

Sur notre logiciel, il est possible d'utiliser quatre algorithmes différents, issus de la bibliothèque scikit-learn (dont nous reparlerons plus tard). Ces algorithmes ont tous des hyperparamètres, qui influencent leur fonctionnement et modifient leurs performances. Ces hyperparamètres sont modifiables par l'utilisateur. Nous allons présenter brièvement les quatre algorithmes en question, d'après l'article [2].

K plus proches voisins (KNN) :

Cet algorithme indique que les points de données dans chaque ensemble de données se trouvent généralement près d'autres éléments ayant des caractéristiques similaires. Lorsque les points de données sont étiquetés, l'étiquette d'un point de données non classifié peut alors être déduite en examinant les étiquettes de ses plus proches voisins. L'algorithme des K plus proches voisins (KNN) identifie les k points de données les plus proches de celui à identifier et détermine son étiquette en trouvant l'étiquette la plus fréquente parmi ces plus proches voisins.

Classification and regression trees (CART) :

Le modèle de classification généré par cet algorithme est structuré comme un arbre avec un nœud racine, des nœuds intermédiaires et des feuilles ; l'arbre généré est appelé arbre de décision (DT). Pour la classification des données, le nœud racine évalue une condition de test pour déterminer la branche appropriée à suivre, et ce processus se poursuit jusqu'à un nœud intermédiaire avec une nouvelle condition de test, ou jusqu'à une feuille associée au résultat de la décision atteinte.

Random Forest : L'algorithme Random Forest (RF) consiste à créer plusieurs arbres de décision (DT

pour Decision Tree) en utilisant un échantillon pré sélectionné à partir des données d'entraînement originales. Chaque DT est généré en utilisant l'algorithme CART. Plutôt que d'utiliser toutes les caractéristiques dans les échantillons, l'algorithme RF adopte une méthode de sélection aléatoire des caractéristiques, qui sélectionne un sous-ensemble de caractéristiques à chaque division de nœud non-feuille dans le DT. Cela est fait pour diminuer la corrélation entre les DT et améliorer la précision.

Multilayer Perceptron (MLP) :

L'algorithme MLP est inspiré du fonctionnement des neurones humains. L'organisation d'un "perceptron" est similaire à l'organisation d'un neurone humain. L'algorithme MLP permet une séparation binaire (séparation en deux groupes) et linéaire (séparation par une droite) d'un ensemble de données.

Pour l'interface, nous avons repris l'affichage de l'étape 2 "modification des données" avec les éléments de paramétrage à droite et le contenu à gauche. Les éléments de configuration du modèle sont à nouveau numérotés pour indiquer l'ordre dans lequel les utiliser. Pour la configuration des hyperparamètres, une autre fenêtre s'ouvre pour afficher les paramètres spécifiques à chaque algorithme.

Après avoir entraîné les algorithmes, plusieurs informations sont affichées sur la page d'entraînement. On peut retrouver le temps d'entraînement de l'algorithme et le temps pour faire les prédictions, ce qui permet à l'utilisateur d'avoir un aperçu de la puissance de calcul requise par chaque algorithme. Une ligne rappelle également quel algorithme a été entraîné, avec les hyperparamètres choisis. Des métriques sont aussi affichées pour apprécier la performance de l'algorithme utilisé. Enfin, une matrice de confusion peut être affichée pour visualiser si l'algorithme effectue les bonnes prédictions (diagonale de la matrice)

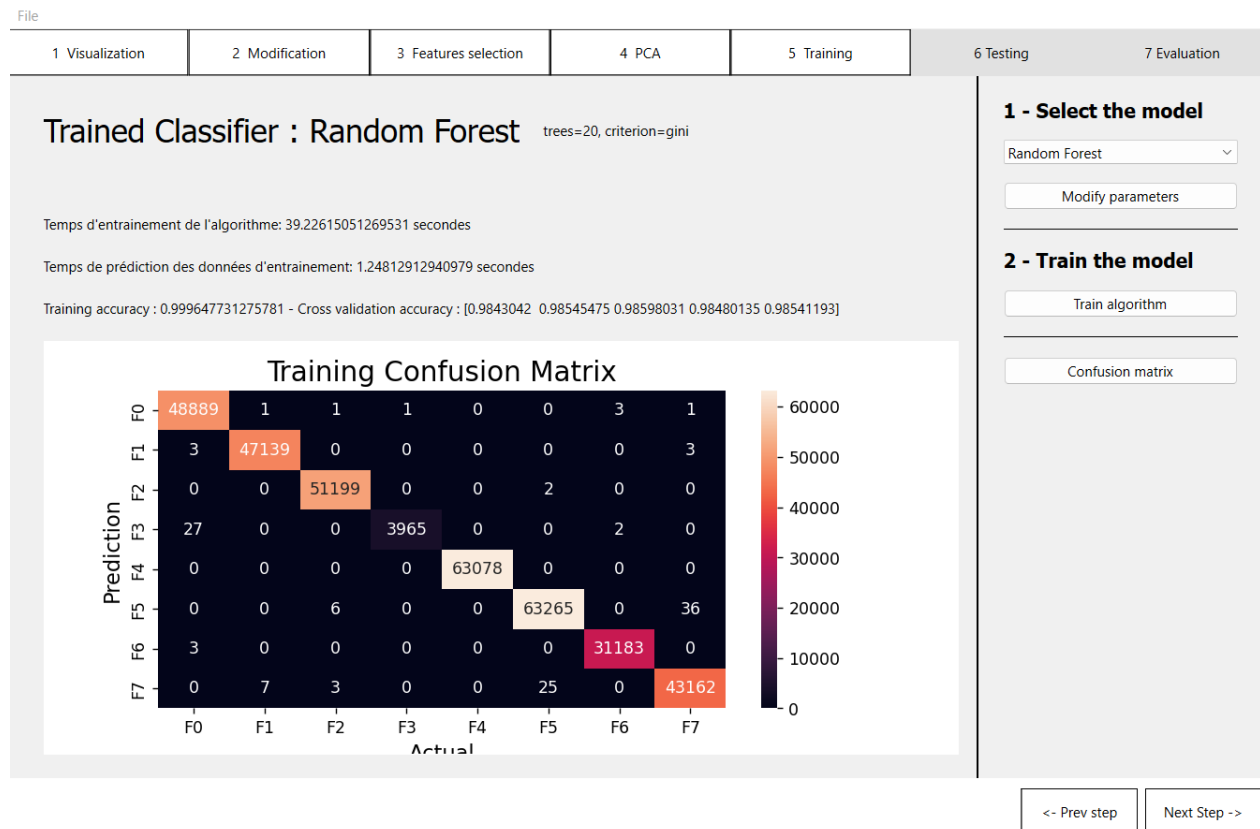


FIGURE 10 – page "training"

Sur la page de test, on retrouve également cette matrice de confusion, avec les données de test cette fois ci. Différentes métriques (F1 score, accuracy, recall score, precision score) sont affichées, permettant de mesurer les performances de l'algorithme configuré précédemment.

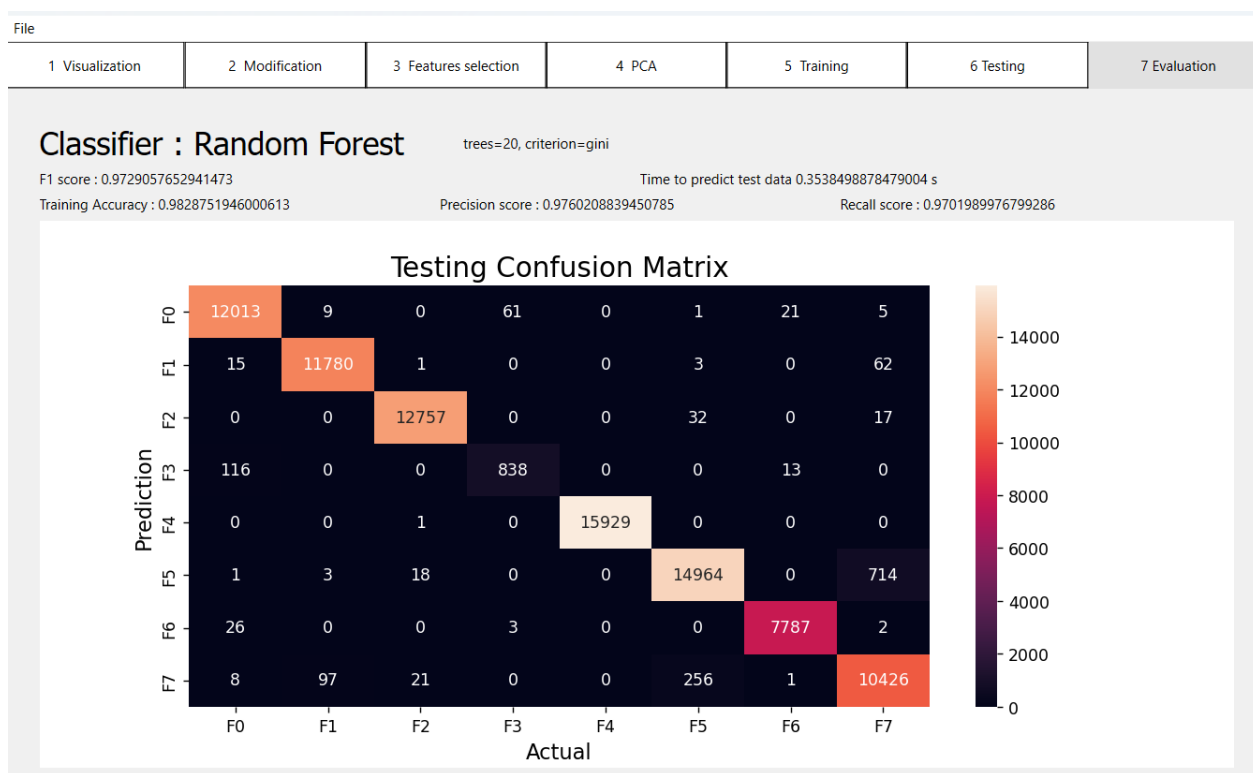


FIGURE 11 – page "testing"

Enfin, la dernière fenêtre est la page d'évaluation dont l'interface se compose de tuiles représentant chacune un des modèles entraînés par l'utilisateur. Son rôle est de permettre l'étude comparative de différents algorithmes. L'utilisateur peut ajouter autant d'algorithmes qu'il souhaite, en choisissant leur type et leurs hyperparamètres. On retrouve pour chaque algorithme les métriques présentées précédemment. Grâce à celles-ci, l'utilisateur peut retenir l'algorithme qui fonctionne le mieux pour le fichier qu'il a importé. Notre logiciel est donc un outil d'aide à la décision. L'objectif est d'exporter les paramètres de l'algorithme pour pouvoir l'utiliser par la suite pour traiter de nouvelles données.

Chaque tuile créée peut être supprimée par l'utilisateur. Il est également possible d'afficher ou de masquer la matrice de confusion pour observer plus en détails les résultats.

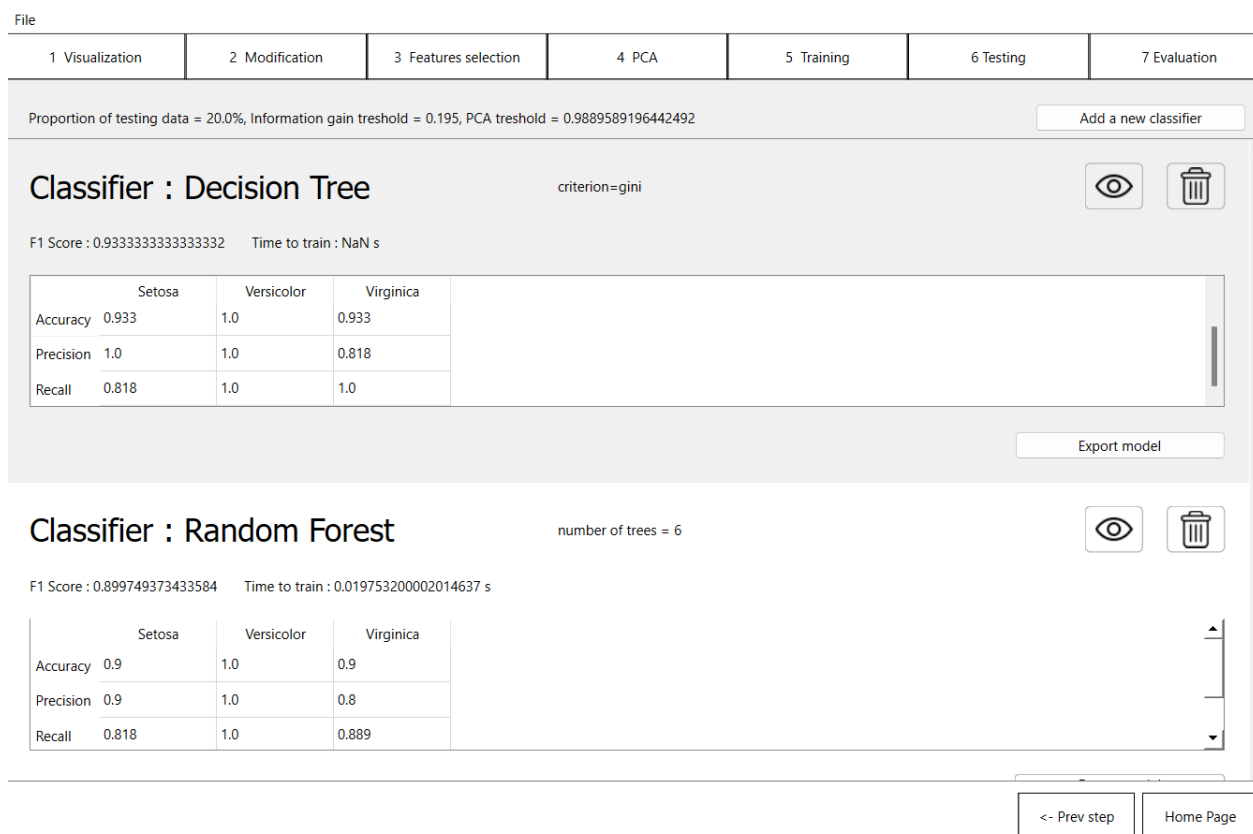


FIGURE 12 – page "Évaluation"

Un bouton d'exportation permet à l'utilisateur d'enregistrer le modèle qui lui semble le plus pertinent. Il a alors le choix de l'enregistrer sous forme d'un fichier .txt ou .pkl. Si il est enregistré en .txt, un fichier est créé contenant le modèle retenu, ainsi que des explications concernant son utilisation, en anglais et en français :

```
FR: Le modèle généré provient de la bibliothèque python sklearn (rendez vous sur le site de sklearn pour obtenir plus
d'informations)
Il contient le modèle, ainsi que les hyperparamètres que vous avez choisis. Si certains hyperparamètres ne sont pas affichés, c'est
que vous avez choisis les hyperparamètres par défaut.
Pour entraîner à nouveau le modèle, utilisez la méthode .fit(X,y) avec X le tableau de données et y le vecteur de labels. Par
exemple: MLPClassifier(learning_rate='invscaling', solver='sgd').fit(X,y) Pour prédire le label d'un jeu de données, utilisez la
méthode .predict(X), avec X votre jeu de données. Par exemple: MLPClassifier(learning_rate='invscaling', solver='sgd').predict(X)

EN: The generated model comes from the sklearn python library (go to the sklearn website for more information)
It contains the model, as well as the hyperparameters that you have chosen. If certain hyperparameters are not displayed, it is
because you have chosen the default hyperparameters.
To retrain the model, use the .fit(X,y) method with X the dataframe and y the associated label vector. For example:
MLPClassifier(learning_rate='invscaling', solver='sgd').fit(X,y)
To predict the label of a given dataset, use the .predict(X) method, with X your dataset. For example:
MLPClassifier(learning_rate='invscaling', solver='sgd').predict(X)

Model: RandomForestClassifier(n_estimators=5)
```

FIGURE 13 – Fichier d'exportation texte

2.3 Les problèmes rencontrés et comment nous y avons fait face

En débutant ce projet, nous n'avions jamais réalisé une interface graphique. Ainsi, il nous a fallu apprendre par nous même les moyens pour en réaliser une, ce qui a constitué la plus grande difficulté de ce projet. Nous avons dû nous familiariser avec de nombreuses bibliothèques Python qui nous étaient étrangères. Le travail de recherche et d'assimilation de nouveaux concepts était difficile, mais nous avons réussi à apprendre et à

mobiliser de nouvelles compétences pour mener à bien ce projet.

En plus de nous familiariser avec les bibliothèques Python, nous avons bien progressé en programmation. Nous avons par exemple découvert la "programmation orientée objet" que nous avons utilisé pour l'entièreté du logiciel. Créer un grand programme implique de structurer le code pour le rendre lisible. Le code a été découpé en 19 sous fichiers organisés avec une architecture logique, voir annexe [15]. Au fur et à mesure, nous avons réécrit le code pour y ajouter les méthodes de programmation nouvellement acquises. Cette étape de réécriture est coûteuse en temps mais est nécessaire pour garder un code propre et logique.

Le second défi que nous avons rencontré est la correction des erreurs de notre code. Elles sont souvent compliquées à détecter et à résoudre. Certaines fois, notre programme fonctionnait, puis le lendemain, sans y avoir touché, il ne fonctionnait plus. Pour résoudre ces problèmes, nous avons souvent utilisé les outils d'intelligence artificielle (chat GPT, Mistral AI) pour détecter les erreurs. Ces IA nous ont fait gagner beaucoup de temps et ont grandement contribué à l'aboutissement du projet. Nous avons également pris connaissance des instructions "try" et "except" qui permettent d'essayer d'exécuter un code, en renvoyant l'erreur si ce n'est pas possible. Dans le meilleur des cas, les erreurs rencontrées apparaissent immédiatement suite à une mauvaise modification du code. Mais il y a aussi les erreurs de nature plus obscure, relatives au fonctionnement interne de l'ordinateur ou du système d'exploitation (une erreur peut apparaître sur un ordinateur et pas sur un autre). Ce type d'erreur était particulièrement compliqué à résoudre, leur cause étant extérieur à la justesse du code. Les bugs et erreurs de code sont souvent difficiles à résoudre et peuvent engendrer de la frustration, mais ils sont intrinsèques à tout projet d'informatique. Nous avons appris à accepter cela, à avoir de la patience pour les résoudre et à utiliser les bons outils. Nous estimons avoir dédié au moins 3 séances de travail à la correction de bugs.

Enfin le dernier problème rencontré est arrivé au milieu de notre projet alors que nous avions développé les principales fonctionnalités du logiciel. Nos tuteurs nous avaient envoyé un fichier contenant de véritables données issues de panneaux photo-voltaïques. Ce fichier était bien plus lourd (plus de lignes et de colonnes) que les fichiers d'entraînement que nous utilisions auparavant. Nous nous sommes alors rendu compte que logiciel, qui fonctionnait parfaitement sur les petits fichiers d'entraînement, ne pouvait pas supporter les "vrais" fichiers. Il nous a donc fallu opérer étape par étape en réécrivant le code du début jusqu'à la fin. Notre erreur avait été d'écrire par nous même des fonctionnalités que Python et les bibliothèques possédaient déjà, ce qu'on appelle les fonctions natives. Ces fonctions natives donnent certes le même résultat que les fonctions que nous avons écrites mais celle-ci sont bien plus rapides (entre 3x et 10x plus rapide). Si la différence est imperceptible pour un petit nombre de calculs, elle est bien visible sur des milliers d'opération.

2.4 Organisation du temps de travail et répartition des tâches

Nous avons débuté notre travail en janvier avec deux présentation du projet par nos tuteurs. Il nous ont conseillé de débuter par la création d'une fenêtre windows avec quelques boutons et du texte. Nous avons créé un prototype en deux semaines (voir annexe 19). Ce prototype nous a permis de nous familiariser avec la bibliothèque PyQt. Une semaine plus tard nous avons ajouté la possibilité d'importer un fichier excel et de l'ouvrir dans la fenêtre, en réponse à notre cahier des charges. Nous avons ensuite ajouté les modèles de machine learning. La majorité du travail consistait à étoffer le logiciel en ajoutant progressivement de nouvelles fonctionnalités.

Notre "groupe projet" a la particularité d'être un binôme. La répartition des tâches a donc sûrement été plus simple à orchestrer pour nous, mais il s'agissait tout de même d'un défi au vu du travail qui nous attendait. Cela nous a poussé à beaucoup nous impliquer dans le projet pour le mener à bien. Le fait que ce projet soit exclusivement numérique a limité les contraintes externes. Nous n'avions en effet pas de contrainte logistique comme l'attente liée à des livraisons de pièces par exemple. Nous seuls étions responsables de notre avancement avec la possibilité de travailler en dehors de Polytech, en plus des sessions de travail prévues les mercredis après-midi et jeudis matin.

Pour mener notre travail à bien, nous avons pris l'habitude d'énoncer clairement nos objectifs à chaque séance et de les diviser en petites tâches (comme le préconisait déjà René Descartes). Cela nous a permis de toujours garder de la motivation et de voir notre projet avancer au fur et à mesure des séances. Une fois les tâches à faire clairement définies, nous nous les répartissions équitablement. Notre logiciel comportant

plusieurs fenêtre et donc plusieurs fichiers, nous travaillions chacun sur un fichier différent, afin d'avancer plus rapidement et d'éviter de résoudre un même problème simultanément. Lorsqu'il n'y avait plus beaucoup de tâches informatiques à effectuer, nous décidions de travailler sur ce rapport ou sur le fichier "read_me", expliquant le fonctionnement du logiciel. Nous avons également pris l'habitude de faire une petite réunion avec notre encadrant doctorant Bassel CHOKR, souvent le jeudi en fin de matinée, pour faire un point sur le projet et fixer de nouveaux objectifs.

3 Apprentissages

3.1 La réalisation d'une interface graphique

Le cœur de notre travail est la réalisation d'une interface graphique. Nous avons donc évidemment du nous former pour la réaliser. Nous avons choisi d'utiliser la bibliothèque Python PyQt6, que nous avons trouvée complète, professionnelle et relativement simple d'utilisation. Nous nous sommes ainsi familiarisés avec plusieurs concepts informatiques. Nous avons découvert et utilisé la programmation orientée objet. En effet, les éléments affichés à l'écran sur notre logiciel sont en grande partie des objets de classes faisant partie de la bibliothèque PyQt6. Une classe définit des caractéristiques, des propriétés, les objets de cette classe en héritent. Par exemple, tous les boutons de notre application sont des objets qui héritent de la classe "QPushButton". Les boutons sont donc "cliquables" et permettent d'activer des méthodes ou des fonctions.

Dans la conception d'interface graphique, nous nous sommes aussi familiarisés avec le concept de "cadre de mise en page" en français. Chacune des interfaces que nous utilisons au quotidien regorge de ces "cadres". Ils permettent de regrouper plusieurs éléments de l'interface entre eux (boutons, textes, ...), afin d'améliorer l'expérience utilisateur et répondre aux besoins fonctionnels.

Lorsque nous échangeons avec nos tuteurs de projet, nous obtenions la liste des fonctionnalités à inclure mais nous étions libres sur la manière de les présenter et de les agencer. L'interface a beaucoup évolué. Nous avons essayé d'améliorer l'ergonomie de l'interface. Pour cela, nous avons adopté une présentation de logiciel "étape par étape", inspirée du configurateur de voiture de Renault (<https://www.renault.fr/vehicules-particuliers/twingo/configurateur.html>). Nous nous sommes également inspirés d'autres interfaces utilisateur comme celles des logiciels de bureautique (type : Microsoft Excel et Word).

Il était important de garder une cohérence graphique entre chaque page afin que l'utilisateur retrouve rapidement les informations aux mêmes endroits. Nous nous sommes donc concertés sur ce point pour pouvoir travailler séparément à partir de règles communes. Nous avons expérimentés plusieurs affichages pour obtenir une bonne lisibilité de l'interface. Les différents leviers d'amélioration de la lisibilité étaient : la gestion de l'alignement du texte, la taille du texte, le regroupement des éléments de navigation en "cadres" et l'espacement entre ces différents éléments. Nous aurions également pu jouer sur les couleurs pour distinguer les éléments importants. Nous ne nous y sommes pas essayés car c'est une solution qui peut, si elle est mal maîtrisée, fatiguer visuellement l'utilisateur. Nous avons conservé une interface simple et épurée.

Nous avons aussi voulu varier les façons d'interagir avec l'interface pour rendre l'utilisation du programme plus "ludique". Pour cela, le logiciel comporte des boutons classiques, des champs d'écriture, des menus déroulants, des curseurs, etc... Pour aider l'utilisateur, nous avons aussi ajouté des textes explicatifs qui apparaissent lorsque l'utilisateur laisse sa souris au dessus d'un bouton.

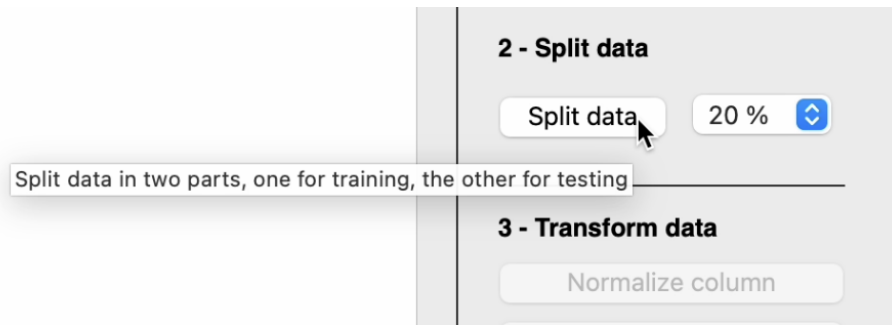


FIGURE 14 – Exemple de texte explicatif

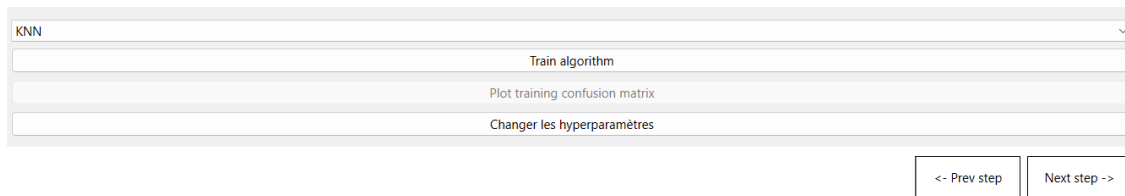


FIGURE 15 – Exemple de boutons

3.2 Introduction aux algorithmes d'IA

Depuis plus d'un an maintenant, l'"Intelligence Artificielle" est en plein essor au près du grand public. L'avènement des IA génératives comme Chat GPT et MidJourney met cette technologie sur le devant de la scène et fait parler d'elle. C'est dans ce contexte que nous avons réalisé ce projet et découvert certains algorithmes ainsi que différents concepts d'IA. Cela nous permet d'avoir un nouveau regard sur cette technologie vaste et dont les applications sont variées.

Pour notre projet, nous avons utilisé une technologie nommée apprentissage supervisé. Elle s'inscrit dans les IA de machine learning, plus spécifiquement de classification. Le but de ces algorithmes est en effet de classer des points de données en leur associant un label (une étiquette). Ce type d'IA est très différent des IA génératives, qui font appel à des concepts de "deep learning" et aux réseaux de neurones, que nous n'avons pas exploré dans le cadre de notre projet.

Nous avons découvert 4 algorithmes (présentés précédemment) mais aussi différentes techniques d'ingénierie des données, nécessaire à leur utilisation. Plusieurs étapes préliminaires sur le jeu de données sont en effet nécessaires avant de pouvoir utiliser les algorithmes. La première est la mise à la même échelle de toutes les caractéristiques, ce qui permet d'éviter la sur-représentation de certaines données. Pour cela, on utilise soit la normalisation (qui fait écho à nos cours de probabilité), soit la standardisation. Ensuite, Les fichiers de mesure provenant des systèmes d'énergies renouvelables contiennent énormément de caractéristiques dont certaines ne sont pas pertinentes pour l'apprentissage des algorithmes. Pour optimiser le temps de calcul et la performance des algorithmes, il est nécessaire de supprimer certaines caractéristiques. Cela peut se faire manuellement, mais aussi et surtout à l'aide de méthodes permettant d'associer un score de pertinence à chaque caractéristique. Pour cela, nous avons utilisé une technique nommée "feature selection" ou sélection de caractéristiques en français. La pertinence des données nous est donnée avec une méthode appelée "information gain". Dans la même optique de simplification des données, l'approche d'analyse en composantes principales (PCA) est également un outil puissant, qui lui fait écho à nos cours d'algèbre car il utilise le concept d'orthogonalité. (D'ailleurs, nous trouvons un lien fort entre l'algèbre et les problèmes de classification car il s'agit de former des groupes de points dans un espace à N-dimensions, avec N le nombre de caractéristiques utilisées. Nous avons apprécié pouvoir nous servir des capacités d'abstraction acquises en algèbre pour mieux comprendre notre sujet.)

3.3 Bibliothèques Python

Ce projet nous a permis de découvrir de multiples bibliothèques Python. Nous avons appris à lire des documentations pour trouver les informations que nous cherchions. Nous avons donc perfectionné nos capacités de recherche d'information, cruciales au travail informatique. La fonctionnalité de recherche "ctrl+f" a été un réel allié pour cela.

Nous avons déjà évoqué la bibliothèque PyQt6 pour la réalisation de l'interface graphique, mais nous en avons utilisé beaucoup plus, 6 différentes au total.

La première, nommée Numpy, est très connue et utile pour les projets informatiques qui nécessitent l'utilisation d'objets mathématiques. Elle permet entre autre de manipuler des matrices et des vecteurs. Elle nous a donc été très utile la partie de traitement de données de notre projet.

La seconde, essentielle à la manipulation de données en Python, est la bibliothèque Pandas. Elle permet de lire des fichiers (csv, xlsx, parquet...) et de les transformer en "dataframe" qui est un objet optimisé pour la manipulation de données. C'est sûrement la bibliothèque qui nous a été le plus utile.

Ensuite, nous nous sommes servi de la bibliothèque Scikit-learn pour toute la partie d'intelligence artificielle, au coeur de notre projet. C'est à partir de cette bibliothèque que nous avons réalisé toute la partie "feature sélection", avec la méthode "information gain". Elle comporte également de quoi réaliser la partie PCA. Tous les algorithmes d'apprentissage supervisé et les hyperparamètres associés qui sont disponibles sur notre logiciel proviennent de cette bibliothèque. Nous avons également utilisé la bibliothèque matplotlib, qui permet d'afficher des graphiques. (vous retrouverez des exemples en annexes)

Enfin, la bibliothèque seaborn a joué un rôle pour la visualisation de données. Elle nous a permis d'obtenir des échelles de couleur pour toutes les matrices qui sont affichées sur notre logiciel (dans les parties de visualisation statistique, d'entraînement des algorithmes, de test des algorithmes et d'évaluation) ce qui rend le résultat plus visuel et parlant.

4 Conclusion

4.1 Critique des résultats obtenus et pistes d'approfondissement

Nous sommes globalement fiers du travail que nous avons réalisé. Le projet était ambitieux, et nous sommes parvenu à un résultat très satisfaisant. Cependant, il reste encore beaucoup à faire pour pouvoir lui donner une application industrielle. Nous avons pour mission de créer un logiciel utilisant des données dites "labelisées" (étiquetées). Cela signifie qu'il faut ajouter manuellement une information sur l'état du système (fonctionnement normal ou défectueux, avec le type de panne associé) pour une grande quantité de points de données. En pratique, quasiment aucune entreprise ne possède ce type de données, ce qui rend contraignant l'utilisation de notre logiciel. Pour outre-passer cette contrainte, nous pourrions explorer les algorithmes d'apprentissage non supervisé (deep learning, réseau de neurones...), qui ne nécessitent pas de données étiquetées.

D'autre part, notre logiciel permet de choisir un modèle de machine learning performant pour détecter les pannes, mais il ne permet pas d'utiliser ce modèle en temps réel. Pour lui donner une application concrète, il faudrait analyser de façon continue et presque instantanée des données provenant des systèmes. L'on pourrait ainsi détecter de potentielles pannes en temps réel et pratiquer la maintenance préventive.

Un autre ajout possible est le calcul des coûts d'entretien. L'utilisateur fournirait au logiciel la liste des coûts d'intervention et de réparation et en fonction des pannes détectées par le logiciel. Il pourrait alors visualiser les économies réalisées en se basant sur les recommandations du logiciel. Nous avons déjà créé les formules de calcul de ces coûts, il ne resterait plus qu'à les implanter dans le code.

Pour finir, nous pourrions développer le logiciel dans plusieurs langues, avec une fonctionnalité permettant de la choisir. Cela lui donnerait une dimension universelle et professionnelle.

4.2 Conclusion générale

En somme, nous avons élaboré une interface graphique utilisateur répondant au cahier des charges fonctionnel présenté. Notre logiciel reprend point par point le travail de recherche de Bassel CHOKR [2], à savoir

la visualisation de données, la préparation des données à l'utilisation d'algorithmes d'IA (séparation des données en deux ensembles, normalisation/ standardisation), l'extraction des caractéristiques principales (avec la méthode "Information Gain") et l'analyse en composantes principales (PCA), l'utilisation des algorithmes de classification et enfin l'étude comparative des performances de chaque algorithme. Le résultat de notre travail est un logiciel utilisable par une personne non-experte, ne sachant pas coder. Il permet de choisir le meilleur algorithme pour des problèmes de classification et d'apprentissage supervisé et s'adapte à tout type de fichiers, à condition que les données soient labélisées. Ce projet était très enthousiasmant pour nous car il a un réel intérêt pratique. De plus, ce travail nous a permis de faire un premier pas dans le vaste domaine du machine learning, à la jonction entre les mathématiques et l'informatique, ce qui est une chance pour nous étant donné que nous nous orientons dans cette voie dans les années à venir. Le produit que nous avons réalisé nous semble assez abouti, nous espérons qu'il pourra trouver une application industrielle. Pour cela, nous avons prévu de contacter des entreprises pour valoriser notre travail. Nous avons d'ores et déjà établi une liste d'entreprises qui pourraient être intéressées par notre projet. Ce projet reste bien sûr perfectible. Si des professionnels se montrent intéressés par notre projet, nous nous montrerons ouverts à toute piste d'amélioration.

4.3 Conclusion personnelle de Louis ARNAUD

J'ai particulièrement apprécié ce projet pour plusieurs raisons. Premièrement, pour son sens. En effet, il était motivant pour moi de travailler pour les énergies renouvelables dans un contexte de changement climatique et d'extinction de la vie sur terre où décarboner l'énergie est un réel enjeu. Deuxièmement, pour les apprentissages et les compétences acquises. J'ai pu découvrir de nouvelles choses en informatique et en mathématiques appliquées, ce qui a été motivant pour moi qui souhaite m'orienter dans cette voie. Enfin, ce projet était pour moi une belle conclusion de ces deux années de classe préparatoire. J'ai pu, grâce à ce projet, mettre en pratique certains apprentissages et la rigueur acquise pendant le début de mes études. Il est très agréable d'avoir réalisé un logiciel qui nous appartient et qui vient essentiellement de nous. Cela m'encourage à continuer de travailler sur ce type de projet et à poursuivre mes études.

4.4 Conclusion personnelle de Noah RAIMBAUD

Je suis fier du projet sur lequel nous avons travaillé. Ce fut l'occasion de découvrir les bases du machine learning, qui est un domaine qui m'intriguait et sur lequel j'aimerais travailler par la suite. Ces projets sont de superbes occasions pour apprendre de par nous-mêmes dans des tâches très diverses : la communication, les réunions hebdomadaires, la planification, la recherche. Ce n'est qu'un aperçu du métier d'ingénieur et j'ai beaucoup apprécié cet aperçu. Le principal point positif est que notre projet avait une réelle utilité en répondant à un besoin de professionnels. C'était une motivation supplémentaire pour créer le meilleur produit possible en espérant qu'il puisse être utilisé. Et même si le résultat n'est pas parfait, la quantité de travail investie dans ce logiciel en fait un superbe support de communication pour montrer nos compétences à des entreprises.

Références

- [1] Contributeurs aux projets Wikipedia. Analyse en composantes principales. https://fr.wikipedia.org/wiki/Analyse_en_composantes_principales, mai 11 2024.
- [2] Bassel Chokr, Nizar Chatti, Abderafi Charki, Thierry Lemenand, and Mohammad Hammoud. Feature extraction-reduction and machine learning for fault diagnosis in pv panels. *Solar Energy*, 262 :111918, 2023.
- [3] IBM Technology. What are mlps (multilayer perceptrons)?, 2022. <https://www.youtube.com/watch?v=7YaqzpBtXw>.

5 Annexes

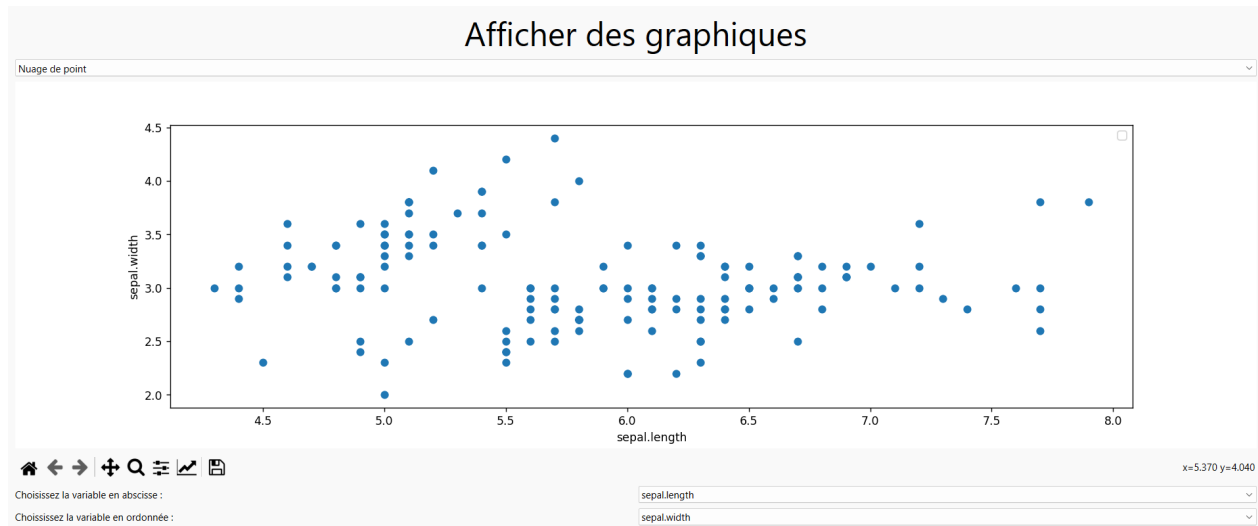


FIGURE 16 – Exemple 1 de graphique matplotlib

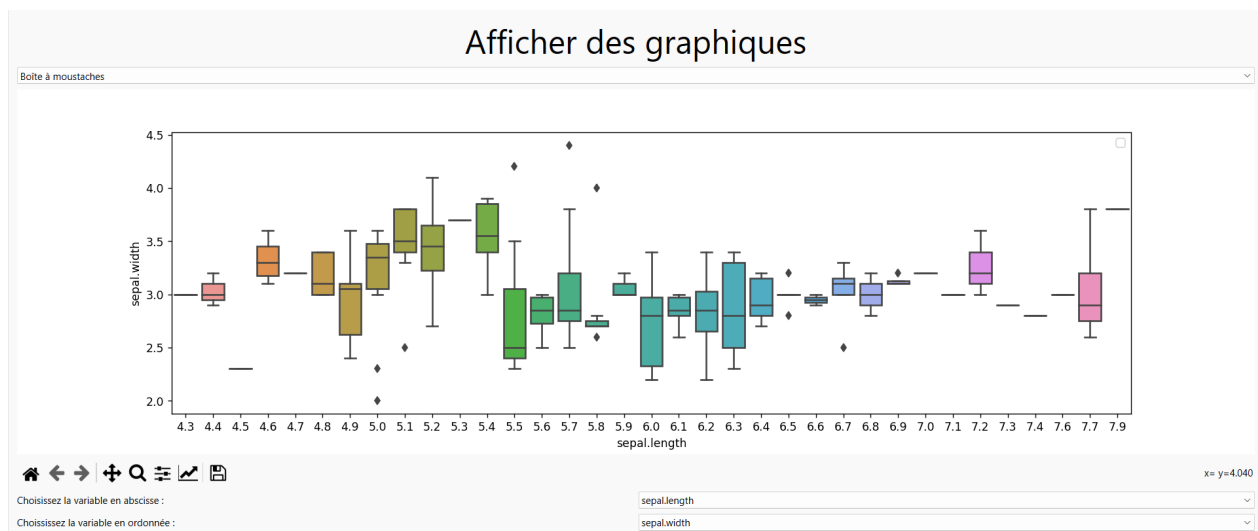


FIGURE 17 – Exemple 2 de graphique matplotlib

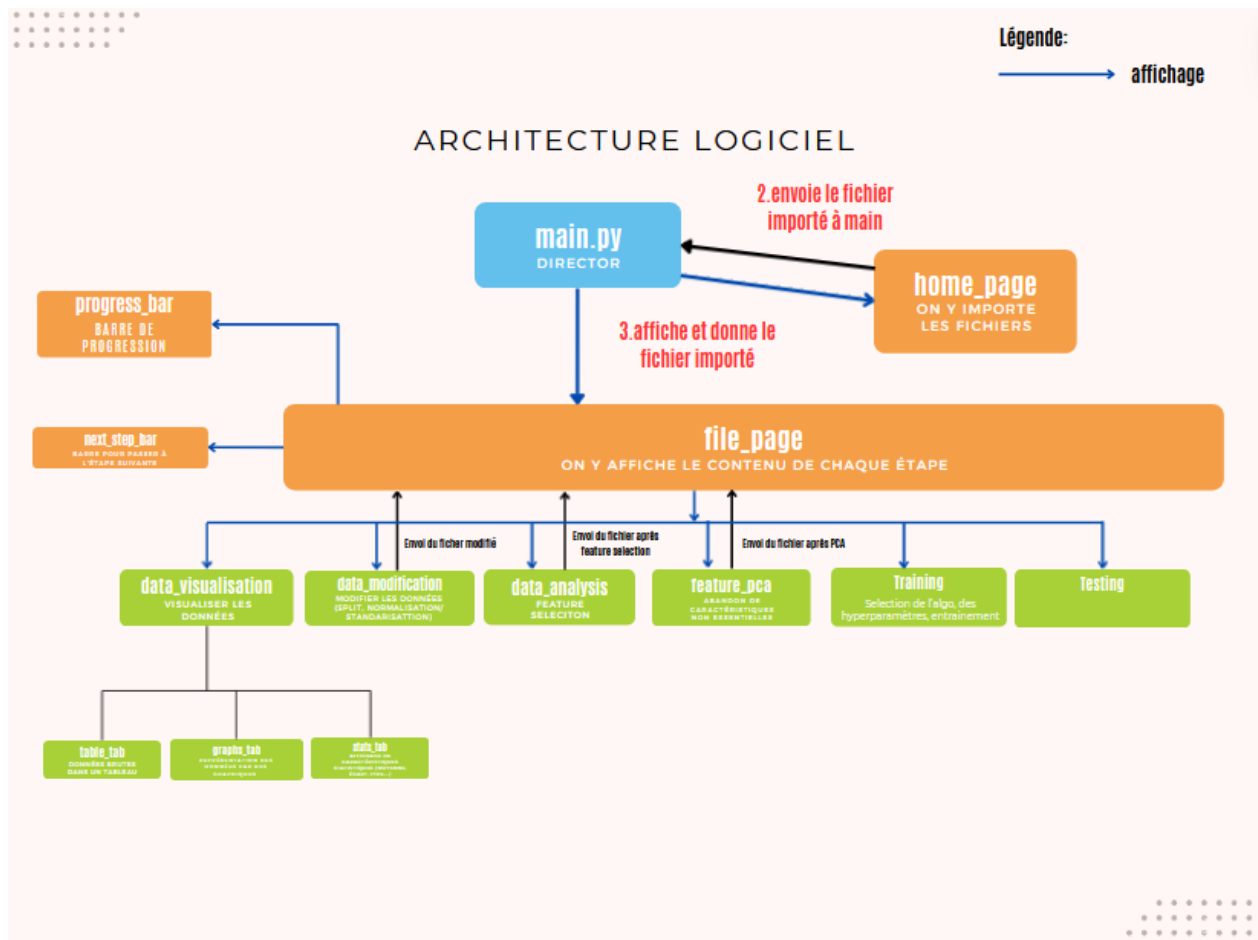


FIGURE 18 – Schéma de l'architecture logiciel

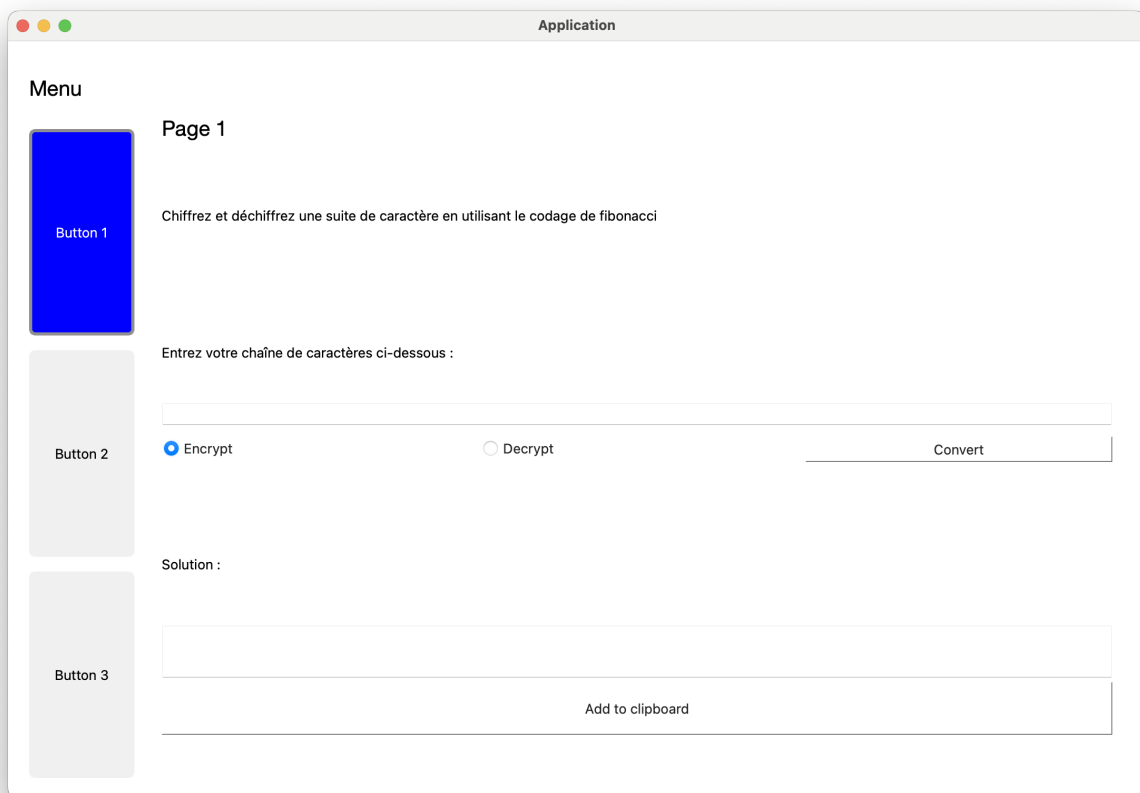


FIGURE 19 – Interface de notre premier prototype